

Since we just covered the high-level strategy—keeping it under 150 lines, separating rules from maps, and strictly defining commands—let's look at exactly what this looks like in practice. When you are managing a complex repository with multiple moving parts, like a branched Open Wiki fork or a custom voice-interaction layer, your CLAUDE.md needs to act as a strict compass.

Here is a blueprint of how to structure the file for an advanced, hardware-specific project:

The Blueprint: CLAUDE.md

Markdown

Project Overview

This is a custom fork of the Open Wiki repo optimized for localized AI and voice interaction. The architecture relies on the Qualcomm QAIRT toolchain targeted for Snapdragon 8 Elite environments.

1. Build and Execution Commands

- **Linting:** ``npm run lint``
- **Model Conversion:** ``qairt-converter --input_network <model.onnx> --output_path <path>``
- **Run Tests:** ``pytest tests/voice_layer/ --disable-warnings``
- **Never attempt to run standard server-side Docker containers; all builds must target the local edge environment.**

2. Hard Architectural Rules

- **Voice Layer Integration:** When modifying the STT (Speech-to-Text) or TTS (Text-to-Speech) pipelines, always ensure audio buffering is handled asynchronously to prevent blocking the main thread.
- **Model Deployment:** All compiled ``.dlc`` (Deep Learning Container) files must be placed strictly in the ``/models/compiled/`` directory. Do not leave raw weights in the repository.
- **Data Formatting:** When prepping local reference data for the agent, rely on the established JSONL caching pipeline. Do not introduce new SQLite dependencies.

3. Code Style & Git Conventions

- **Commit Messages:** Must follow the ``type(scope): description`` format (e.g., ``feat(tts): optimize audio chunking``).
- **Python:** Strictly adhere to PEP 8; use type hinting for all new function definitions.

4. Context Pointers (Index)

If you need deeper context, read the following files:

- QAIRT configuration and GenieX pipeline details: ``@docs/qairt-toolchain.md``

```
- Voice layer (STT/TTS) latency requirements: `@docs/voice-latency-specs.md`  
- Core Open Wiki upstream sync rules: `@docs/upstream-sync.md`
```

Why This Structure Works

1. **Immediate Boundary Setting:** By explicitly mentioning the hardware target (Snapdragon 8 Elite) and the toolchain (QAIRT) right at the top, the lead agent instantly drops generalized web-development assumptions and shifts into edge-computing mode.
2. **Actionable Guardrails:** Notice the rule about not using standard Docker containers. If the agent ever tries to write a standard docker-compose file to fix a build issue, this rule stops it and forces it to use your edge-specific deployment flow.
3. **Delegated Memory:** By isolating the heavy toolchain documentation into `@docs/qairt-toolchain.md`, your main `CLAUDE.md` stays lean. The agent will only spend tokens reading about the toolchain when you specifically ask it to debug a model conversion.